

主存与存储硬件

一个地址怎样落到芯片、阵列、介质与可计量的事务

408 · 计算机组成原理 Topic CO-05

母问题：抽象地址怎样被有限物理硬件兑现？

一个存储请求至少要回答四件事：**地址代表什么、由谁保存、一次怎样读写、多久能完成/再次服务**。本册用

Address / Granularity → Module / Chip / Bank Selection
→ Array / Medium Operation → Transfer Transaction
→ Latency / Cycle / Throughput / Bandwidth

组织知识。箭头表示**实现依赖与分析顺序**，不是说所有介质都具有相同内部时序。

本册负责什么

- **Own**：编址单位、存储字、SRAM/DRAM/Flash/HDD 的硬件组织、芯片扩展、地址译码、Bank/交叉编址、刷新和介质访问事务；性能语言包括 latency、access time、cycle time、throughput 与 bandwidth。
- **Use / Output**：向 CO-06 输出 Cache miss 所需的下一级服务成本，向 CO-08 输出存储端事务约束；磁盘调度、文件映射、OS Page Cache 与页面置换归 OS。
- **Stop Boundary**：出现 Cache tag/valid/replacement 时转 CO-06；出现 VPN/PFN/TLB 时转 CO-07；出现文件/进程调度策略时转 OS Owner。

目录

1 先建立坐标：同一个“存储器”可以按不同分类轴观察	3
2 第一原则：先分粒度，再做容量与地址线	3
3 信息怎样被物理单元保存：SRAM、DRAM 与非易失介质	4
3.1 SRAM 与 DRAM：保持方式生成访问代价	4
3.2 DRAM 一次访问的状态轨迹	4
3.3 刷新：保持数据的维护事务	4
3.4 Flash / SSD：写和擦除不是同一粒度	4
4 芯片扩展与地址译码：容量是“深度 × 宽度”两项预算	5

5 多模块与交叉编址：提高的是可重叠服务能力，不是让单体变快	5
6 性能语言：Latency、Access Time、Cycle、Throughput 与 Bandwidth	6
7 HDD 与块设备：接口地址、物理几何和访问成本分层	7
8 贯穿母例：从芯片规格到一次连续访问	7
9 概念边界	8
10 做题控制协议	8
11 本册与其他 Owner 的接口	9

1 先建立坐标：同一个“存储器”可以按不同分类轴观察

同一个器件可以同时属于多个类别，分类前必须先声明问题轴：

分类轴	典型类别	真正回答的问题	易错点
存取方式	random / sequential / direct / associative	地址给出后怎样定位数据	把 random 理解成零延迟
物理介质	SRAM / DRAM / Flash / 磁 / 光	信息由什么物理状态保持	把介质名当系统层次名
易失性	volatile / non-volatile	断电后是否保持信息	把刷新与掉电保持混为一谈
系统位置	register / Cache / main memory / storage	在容量、速度、成本中承担什么角色	认为同一介质只能承担一种角色

核心术语

- **编址单位**：一个地址直接命名的最小信息单位；按字节编址时通常为 1B。
- **存储字**：阵列内部一次组织/选择的数据宽度，不必等于 CPU 字长或总线宽度。
- **Bank**：可与其他 Bank 部分独立工作的存储阵列/模块；并行能力仍受控制器、总线和冲突限制。
- **Transaction**：接口层一次完整请求/响应过程，可能包含地址、命令、等待和一个或多个数据 beat。

2 第一原则：先分粒度，再做容量与地址线

最容易被混为一谈的至少有：

- 编址单位：一个地址命名多少信息；
- 存储字：阵列一次组织多少位；
- 芯片数据宽度：单片一次贡献多少位；
- 总线/事务宽度：一次 beat 或一次事务搬多少；
- 上层批量粒度：Cache line、虚存 page 等。

这些量可以相等，但不能因为都写成 B/bit/word 就默认相等。

若共有 N 个可寻址单元、每单元 u bit，则

$$Capacity = N \times u.$$

n 条地址线最多区分 2^n 个**编址单元**。因此地址线由“有多少个地址”决定，而不是由 CPU 字长或总字节数单独决定。

3 信息怎样被物理单元保存：SRAM、DRAM 与非易失介质

3.1 SRAM 与 DRAM：保持方式生成访问代价

SRAM 用双稳态反馈保存位，通常读不破坏、无需周期刷新，但单元面积大、密度低，适合高速小容量层。DRAM 用电容电荷保存位，密度高、成本低，却因电荷泄漏需要刷新，读出通常还涉及感测与恢复，适合作为大容量主存。二者通常都易失；*static* 只表示无需周期刷新，不表示断电保存。

维度	SRAM	DRAM	做题信号
保持机制	双稳态反馈	电容电荷	为什么刷新/为什么密度不同
读操作	通常非破坏	感测后恢复	是否有恢复时间
阵列组织	直接地址选择	行激活、列选择、row buffer	RAS/CAS、行命中
典型角色	Cache 等高速小容量存储	主存	速度/容量/成本权衡

3.2 DRAM 一次访问的状态轨迹

典型依赖可写成

row address → activate / sense → column select → data transfer → restore / precharge.

DRAM 常复用同一组外部引脚分时传送行地址和列地址，减少引脚；行激活后整行进入行缓冲，同一已激活行的后续列访问可能更快。不同题设会抽象掉部分阶段，因此不能把所有 DRAM 题固定成同一拍数。

若阵列有 r 行、 c 列，行列地址复用，地址引脚数至少满足

$$A_{pin} = \max(\lceil \log_2 r \rceil, \lceil \log_2 c \rceil).$$

只有对称阵列时才可把总地址位数简单平分。

3.3 刷新：保持数据的维护事务

刷新由存储系统/控制器组织，不是 CPU 把每个字重新写回。若有 r 行、每一行都必须在保留期限 T 内至少被刷新一次，则均匀异步调度的最大平均刷新间隔满足

$$\Delta t \leq \frac{T}{r}.$$

因此 T/r 是 deadline 边界，不是“每次刷新必须精确相隔这么久”的等式：更早刷新仍然合法。只有题目问“最多能插入多少次普通读写”时，才需要把调度尽量推近这个上界。集中刷新把刷新工作聚集成一段不可访问时间；分散刷新把刷新插入访问周期；异步刷新把各行维护事件摊开。三者主要区别是停顿分布和调度方式，不改变“DRAM 必须在期限内周期恢复电荷”的事实。

3.4 Flash / SSD：写和擦除不是同一粒度

Flash 以电荷/阈值状态保存信息，通常页级读写、块级擦除，因此不能像 DRAM 一样任意原地改写。SSD 在 Flash 之上提供主机可见的块设备接口，并通过映射表、垃圾回收、磨损均衡等机制管理介质；这些机制可能产生写放大和延迟波动。

4 芯片扩展与地址译码：容量是“深度 × 宽度”两项预算

若单片为 $D \times W$ ，目标为 $D_t \times W_t$ ，则常用预算为

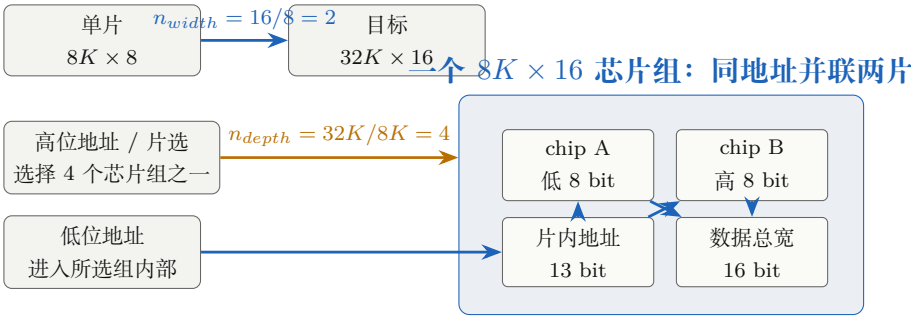
$$n_{width} = \left\lceil \frac{W_t}{W} \right\rceil, \quad n_{depth} = \left\lceil \frac{D_t}{D} \right\rceil, \quad n = n_{width} n_{depth}.$$

位扩展让多个芯片并联贡献同一地址的数据位；字扩展用高位地址/片选选择不同芯片组。一般先补足目标数据宽度，再复制足够多的地址深度组。位扩展完成后，芯片规格中的“ D 个字位置”还必须换算到系统的全局编址单位：若目标字长为 W_t bit、系统按字节编址，则一个 $D \times W_t$ 芯片组覆盖 $D \times W_t/8$ 个字节地址；若按字编址，则覆盖的仍是 D 个可寻址字。芯片深度、目标字长与 CPU 编址单位不能混成一个数量。

全译码使用所有需要的高位地址，地址唯一；部分译码忽略部分高位，电路更省但可能产生地址重影；线选可能浪费地址空间。“这个地址能读到数据”不等于“它只对应这一个物理单元”。

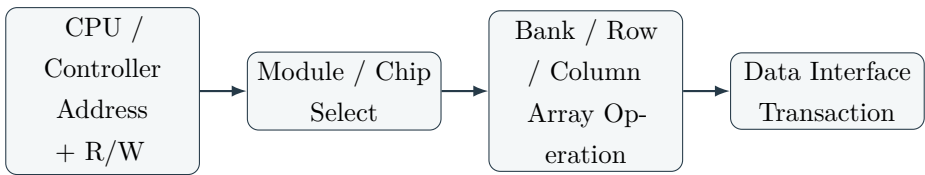
芯片扩展：先做位扩展凑字长，再做字扩展凑地址深度

例：单片 $8K \times 8$ ，目标 $32K \times 16$ 。位扩展需要 2 片并联；字扩展需要 4 组，因此总共 8 片。若系统按字节编址，一组 $8K \times 16$ 覆盖 $16KB$ 地址跨度。



地址职责分层：位扩展不增加地址深度，同一地址让多片同时工作以拼宽数据；字扩展才由更高地址位/片选在多个芯片组之间选择。先形成目标字长，再把“每组有多少个存储字”换算成系统的全局编址跨度。

禁止误读： $8K$ words 不等于 $8KB$ 。在本例中每个目标存储字为 16 bit，所以一组 $8K \times 16$ 对应 $16KB$ 的字节地址空间。



图中的箭头表示**地址解释与服务依赖**：高低地址位怎样分工、行列是否复用、Bank 是否并行都服从具体器件和题设。

5 多模块与交叉编址：提高的是可重叠服务能力，不是让单体变快

低位交叉常把连续地址轮转到不同 Bank；若交叉粒度就是一个编址单元，常见抽象为

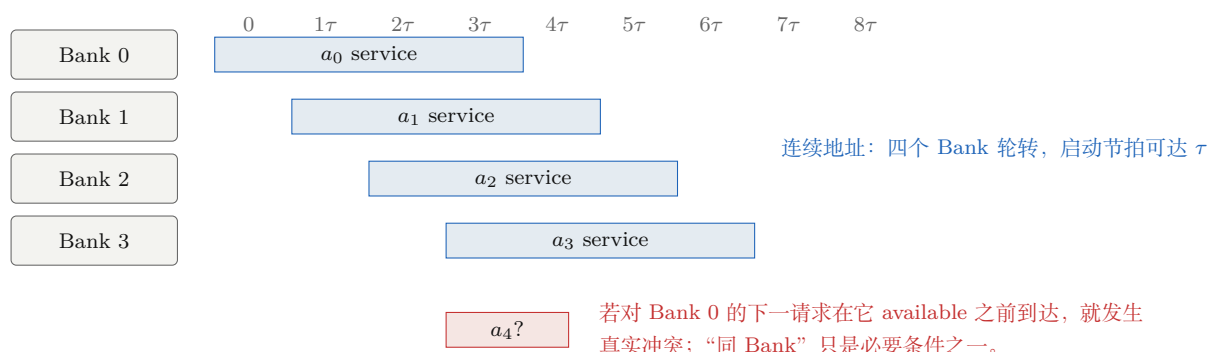
$$bank(a) = a \bmod m, \quad local(a) = \left\lfloor \frac{a}{m} \right\rfloor.$$

若按字、块或更大粒度交叉，应先把地址换成对应粒度编号再取模。高位交叉更像把大段连续地址分区到不同模块，不能自动获得顺序流的 Bank 并行。

两个请求落到同一 Bank 只是**潜在冲突条件**；是否真正冲突还取决于到达间隔和单体忙时间。

低位交叉：把连续请求轮转到不同 Bank，购买的是可重叠服务能力

例：4 个 Bank，交叉粒度为 1 个编址单元， $bank(a) = a \bmod 4$ 。设单 Bank 忙时间 $T = 4\tau$ ，控制器每隔 τ 可尝试启动下一请求。



性能读法：首个请求的结果仍要等一个 Bank 的服务 latency；多体交叉主要把后续请求的启动窗口错开，使稳态 throughput/bandwidth 接近每 τ 一个结果。若 $T > m\tau$ 、总线更慢或到达流重复撞同一 Bank，吞吐会被新的瓶颈限制。

重复 burst 不能只套首轮 $T + (q-1)\tau$ ：跨轮时继续维护各 Bank 的 available time，轮间相位也是状态。

设单体存储周期为 T ，连续请求的传送/启动节拍为 τ ，有 m 个体且无冲突；在经典流水模型满足 $T \leq m\tau$ 时，单个不间断连续流读 q 个字可写成

$$t_q = T + (q-1)\tau.$$

但若题目把一个有限长度 burst 重复多次，并规定每轮都从同一 Bank 相位重新开始，则轮与轮的边界本身也是状态：上一轮最后几次 Bank 启动可能尚未越过 T 的 busy window。此时“本轮有 q 个字”不能直接推出“下一轮在 $q\tau$ 后无条件开始”，应继续维护各 Bank 的 available time。首轮 latency、稳态轮间 initiation interval 与最后排空时间必须分别记账。

若每字为 W B，稳态数据率在瓶颈允许时约为

$$B_{steady} = \frac{W}{\tau},$$

临界 $T = m\tau$ 时也可写为 mW/T 。这说明单体首个访问延迟未必下降，改变的是多个体工作窗口的重叠和稳态服务能力。

6 性能语言：Latency、Access Time、Cycle、Throughput 与 Bandwidth

这些词都在说“快慢”，但对象不同：

- **Latency：**一般意义上的请求到所需结果可用的等待；
- **Access Time：**某存储器件/接口完成题设规定访问的时间，必须声明起止点；
- **Cycle Time：**同一资源下一次独立操作可以合法启动的最小间隔；
- **Throughput：**单位时间完成的请求/事务数；
- **Bandwidth：**单位时间传输的有效数据量；
- **Burst：**一次地址/命令阶段后连续传送多个数据 beat，以摊薄控制开销。

只有在每个事务有效数据量固定且统计口径一致时，才可近似写

$$Bandwidth \approx Throughput \times BytesPerTransaction.$$

“数据宽度 × 频率”也只有在每周期传输次数、协议效率、等待周期和利用率明确时，才是有效带宽。

性能题最容易混淆的三件事

更高 throughput / bandwidth 不自动推出更低单请求 latency；更短 access time 不自动等于更短 cycle time；多个 Bank 也不能消除同 Bank 冲突。

7 HDD 与块设备：接口地址、物理几何和访问成本分层

机械磁盘访问时间在经典题中常分为

$$T_{access} = T_{seek} + T_{rotation} + T_{transfer} + T_{controller}.$$

若未给具体角位置，平均旋转等待常近似半圈：

$$T_{rotation} \approx \frac{1}{2} \frac{60}{RPM}.$$

盘面/磁道/扇区是物理几何；LBA 是主机看到的线性块编号。固件、坏块重映射、分区和缓存等都意味着连续 LBA 不必对应连续真实物理位置。磁盘调度（FCFS、SSTF、SCAN 等）归 OS；CO-05 只提供介质本身的机械和传输成本。

8 贯穿母例：从芯片规格到一次连续访问

给定目标主存容量、单片规格、CPU 编址方式和总线宽度，完整推理顺序是：

1. 写清编址单位与目标“可寻址深度 × 每地址数据宽度”；
2. 分别算 width expansion 与 depth expansion；
3. 划分片内地址、片选地址、数据线和读写控制；
4. 若为 DRAM，继续标 row/column、感测/恢复与刷新；
5. 若为多 Bank，写 bank function 与请求间隔，判断是否真正冲突；
6. 最后分别计算单次 latency/access time、下一请求合法间隔、throughput 与 bandwidth。

这个母例把容量、地址路由、硬件状态和性能放在同一条链中，又不会把它们压成一个公式。

9 概念边界

边界	为什么容易混淆	真正判据 / 题面信号	混淆后果
编址单位 ≠ 存储字	都描述 “一个单位”	地址命名粒度 vs 阵列组织宽度	地址线/容量计算错
存储字 ≠ 总线事务	宽度常相同	阵列组织 vs 接口传输	错算一次事务数量
Access Time ≠ Cycle Time	都用 ns	结果何时可用 vs 下一操作何时可启动	漏恢复/重叠条件
Latency ≠ Bandwidth	都描述性能	单请求等待 vs 单位时间数据量	用带宽替代随机访问延迟
Throughput ≠ Bandwidth	都是稳态指标	requests/s vs B/s	单位与事务粒度错误
Bank 同号 ≠ 必然冲突	都共享同一 Bank	还要看时间上是否重叠	只按取模值判冲突
Cache line ≠ DRAM row	都是批量数据	上层副本粒度 vs 介质激活粒度	把 Cache hit 与 row hit 混写
LBA ≠ 真实物理位置	都是 “块号”	主机接口编号 vs 介质内部布局	由连续 LBA 推出连续盘面

10 做题控制协议

1. **对象：**一个地址代表 byte、word 还是其他单位？一次请求要多少数据？

2. **组织：**芯片深度/宽度、Bank 数、row/column、总线宽度分别是什么？

3. **路由：**哪些地址位做片内地址、片选、Bank 号、行列选择？

4. **事务：**完成目标对象需要几次存储/总线事务？是否跨边界？

5. **时间：**分别给出 latency/access time、cycle time、throughput、bandwidth；检查可否重叠和冲突；

6. **验证：**用最小/最大地址、连续地址、同 Bank 地址和单位量纲做独立检查。

第一动作

看到容量、芯片扩展、交叉编址或主存性能题，先写：“一个地址命名什么？一次阵列操作多少位？一次事务搬多少？下一次请求何时合法？”四个问题没分开之前，不开始套公式。

11 本册与其他 Owner 的接口

相邻 Owner	CO-05 输出什么	何时转交
CO-06 Cache	下一级访问/传输服务成本、带宽上限	开始讨论 tag/valid/replacement/AMAT
CO-08 总线与 I/O 硬件	存储端宽度、事务与可重叠条件	开始讨论仲裁、DMA、控制器协议
CO-07 地址翻译	物理存储器的可访问接口	开始讨论 VPN/PFN/TLB/page walk
OS 存储/I-O	介质访问成本与设备接口约束	开始讨论调度、文件映射、Page Cache、换页策略

一页压缩

Address / Granularity → Chip / Bank / Row-Column
→ Physical Operation / Transaction
→ Latency / Cycle / Throughput / Bandwidth

忘掉具体芯片公式时，从“地址命名粒度、深度/宽度预算、硬件路由、事务数量、时间/速率”即可重新生成主干答案。